

Part IV – Engineering Sprint

Building Business Logic with Apex

“Engineering is learned by building, not by watching.”

Sprint Objective

You are now a member of the Salesforce Development Team at **VIT Solutions Pvt. Ltd.**

Your client, the Placement Office, has approved the system design and is eager to see the first working version of the application.

Today’s sprint has one objective.

Implement the business logic that allows students to apply for jobs correctly.

You are no longer discussing ideas.

You are now building software.

Sprint Backlog

Your team has committed to completing the following User Stories during this sprint.

Story ID	User Story	Priority
US-1	Accept a student application.	High
US-2	Prevent duplicate applications.	High
US-3	Check application deadline.	High
US-4	Verify student eligibility.	High
US-5	Save the application successfully.	High
US-6	Display meaningful feedback to the user.	Medium

Complete every story in sequence.

Do not move to the next story until the previous one has been reviewed by another pod member.

Engineering Sprint 1

Creating the Application Service

Business Requirement

The Placement Office wants one dedicated service responsible for handling all application-related operations.

Think Like an Engineer

Before writing any code, discuss the following questions.

- Why should application processing be separated from student registration?
- If another developer joins the project six months later, where would they expect application logic to exist?
- Would creating multiple unrelated responsibilities inside one class make future maintenance easier or harder?

Discuss for five minutes before opening your IDE.

Coding Task

Create a class named **ApplicationService**.

At this stage, the class should only contain the structure required to support future business operations.

Do not worry about implementation yet.

Focus on creating a clean foundation.



Expected Outcome

By the end of this exercise your project should contain a dedicated service that clearly represents one business responsibility.

Peer Review Checklist

Your reviewer should verify the following.

- ✓ Appropriate class name.
 - ✓ Clear business responsibility.
 - ✓ No unnecessary code.
 - ✓ Good formatting and readability.
-

Engineering Sprint 2

Receiving an Application

Business Requirement

A student selects a company and clicks **Apply**.

The software should receive the request and begin processing it.

Think Before Coding

Discuss with your pod.

What minimum information should the software receive?

Possible discussion points include

- Student Identifier
- Job Identifier
- Date of Application

Should anything else be required?

Can unnecessary information create confusion?

Coding Task

Create a method named

submitApplication()

Accept the required parameters.

At this stage, simply display a success message indicating that the request has been received.

Do not perform validations yet.

Build incrementally.

Expected Behaviour

The software successfully receives an application request.

No business validation has been added yet.

Engineering Principle

Build one capability at a time.

Professional developers avoid solving five problems simultaneously.

Engineering Sprint 3

Preventing Duplicate Applications

Business Requirement

A student should not be allowed to apply for the same company more than once.

Think Like a Consultant

Before writing Apex, discuss.

How can software determine whether the application already exists?

Where should this validation occur?

Before saving?

After saving?

Why?

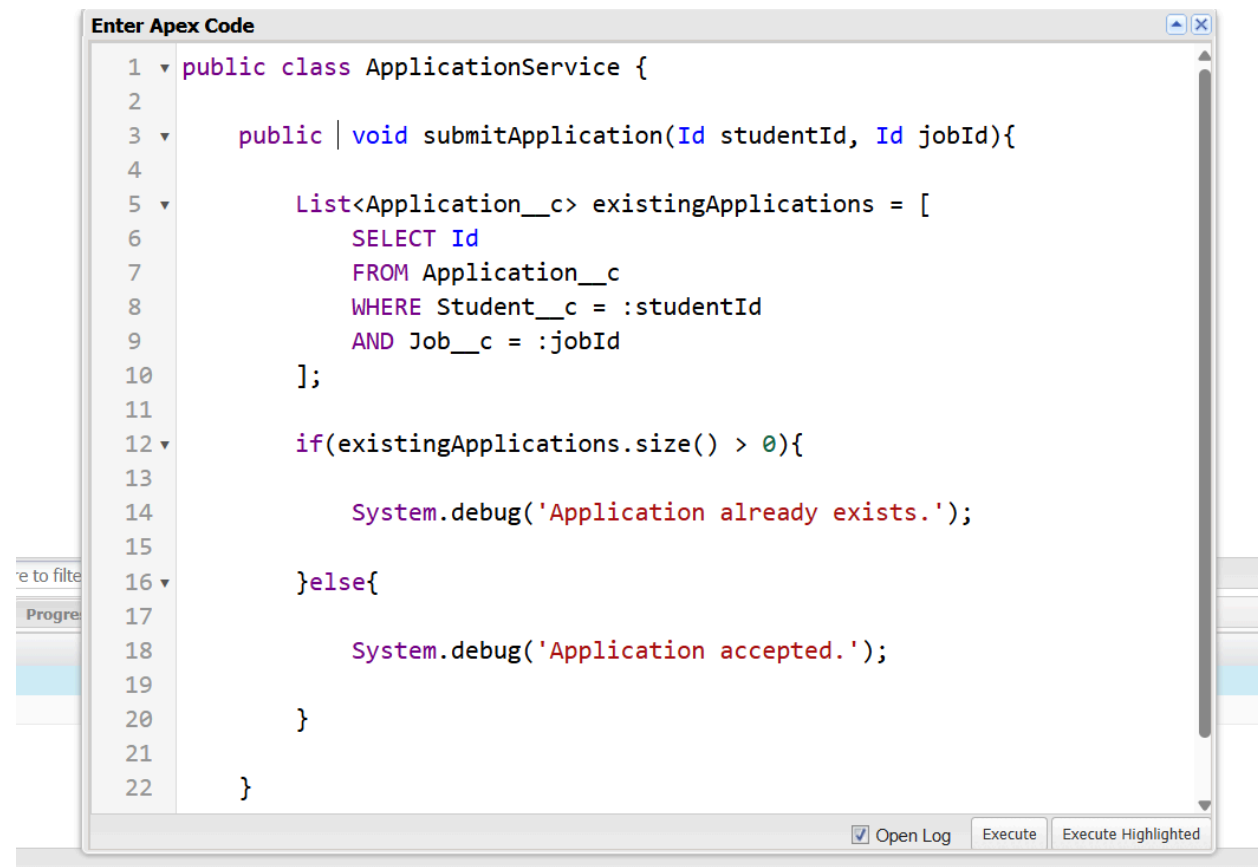
Coding Task

Modify **submitApplication()** so that it first checks whether an application already exists.

If a duplicate is found,

display an appropriate message.

Otherwise,



```
1 public class ApplicationService {
2
3     public void submitApplication(Id studentId, Id jobId){
4
5         List<Application__c> existingApplications = [
6             SELECT Id
7             FROM Application__c
8             WHERE Student__c = :studentId
9             AND Job__c = :jobId
10        ];
11
12        if(existingApplications.size() > 0){
13
14            System.debug('Application already exists.');

The screenshot shows a code editor window titled "Enter Apex Code" with a line number margin on the left (1-22) and a scrollbar on the right. The code implements the submitApplication method in the ApplicationService class. It uses a SOQL query to check for existing applications with the same student and job. If found, it logs a debug message; otherwise, it logs that the application is accepted. At the bottom right, there are buttons for "Open Log", "Execute", and "Execute Highlighted".


```

Test Scenarios

Scenario	Expected Result
First application	Accepted
Second application for same job	Rejected
Same student applying for another company	Accepted

Peer Review

Can another developer understand your validation logic within one minute?

If not,

simplify it.

Engineering Sprint 4

Validating Eligibility

Business Requirement

Companies specify eligibility criteria.

Only eligible students should be allowed to apply.

Examples include

- Minimum CGPA
 - Active Backlogs
 - Branch
 - Graduation Year
-

Think Like an Engineer

Which validations belong together?

Should all validation logic remain inside one large method?

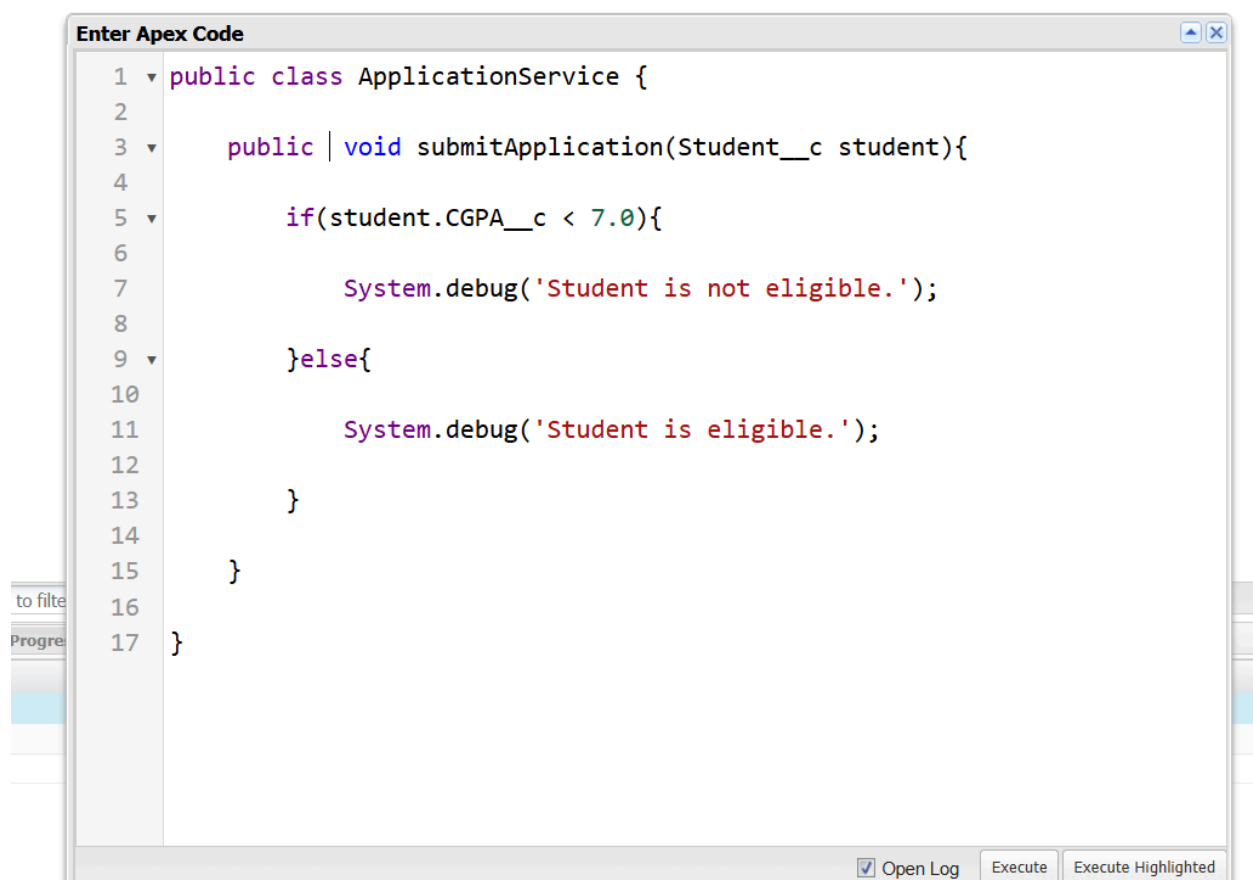
Would smaller helper methods improve readability?

Discuss before coding.

Coding Task

Extend **submitApplication()** to validate the required eligibility conditions.

Return meaningful messages whenever a condition is not satisfied.



```
1 public class ApplicationService {
2
3     public void submitApplication(Student__c student){
4
5         if(student.CGPA__c < 7.0){
6
7             System.debug('Student is not eligible.');

to filter  
Progress



☒ Open Log   Execute   Execute Highlighted


```

Challenge

How would your design change if every company had different eligibility rules?

Discuss before implementation.

Engineering Sprint 5

Saving the Application

Business Requirement

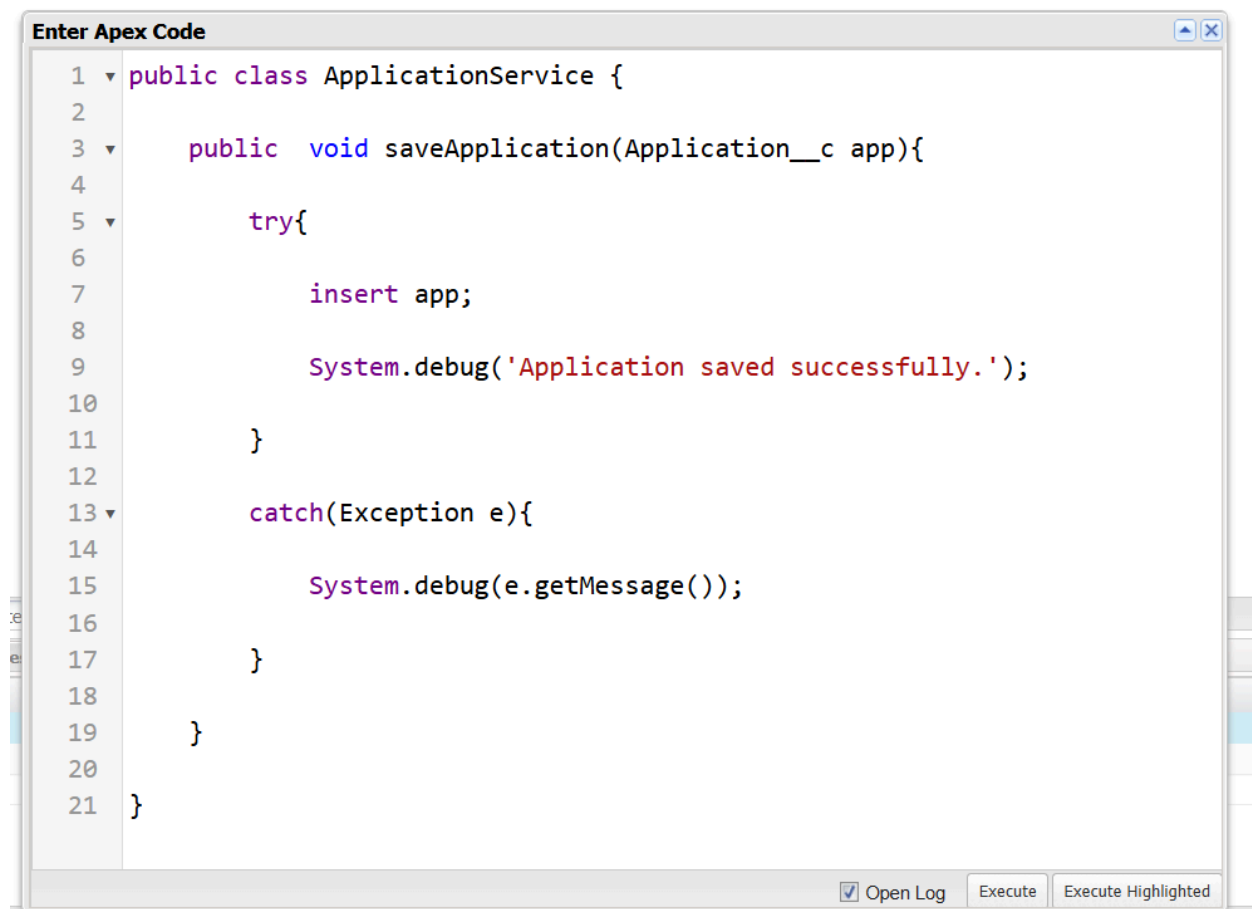
Once every validation succeeds,
the application should be stored in Salesforce.

Coding Task

Use DML to save the application record.

Handle situations where saving fails.

Ensure the user receives an informative message.



```
1 public class ApplicationService {
2
3     public void saveApplication(Application__c app){
4
5         try{
6
7             insert app;
8
9             System.debug('Application saved successfully.');

Open Log Execute Execute Highlighted


```

Think Beyond the Code

What happens if Salesforce cannot save the record?

Should the user simply see

“Something went wrong.”

Or should the software provide a more meaningful explanation?

Good software communicates clearly.

Engineering Sprint 6

Completing the Workflow

At this point,

your software should perform the following sequence.

Receive Application

↓

Validate Duplicate

↓

Validate Eligibility

↓

Save Record

↓

Display Result

If your implementation follows this sequence,

you have successfully built your first business service.

Congratulations.

Debug This!

Every software engineer spends a significant part of the day debugging.

Learning to recognise poor code is as important as learning to write good code.

Study the following situations.

Situation 1

A SOQL query is written inside a loop.

Discuss.

Why might this become a problem when hundreds of students apply simultaneously?

How can the code be improved?

Situation 2

The same validation logic appears in three different methods.

Discuss.

What maintenance problems could arise?

How would you redesign the solution?

Situation 3

Method names such as

`doWork()`

`execute()`

`process()`

appear throughout the project.

Discuss.

Can another developer understand their purpose without opening the implementation?

Suggest better names.

Situation 4

A method contains more than two hundred lines of code.

Discuss.

How would you divide this responsibility into smaller methods?

Interview Corner

During interviews, recruiters often evaluate how you think rather than how much syntax you remember.

Discuss the following questions with your pod.

Question 1

Why should business logic be implemented in service classes rather than inside the user interface?

Ans :It keeps the code clean, reusable, and easy to maintain.

Question 2

What advantages does separating responsibilities provide?

Ans :It makes the application easier to understand, test, and update.

Question 3

What characteristics make a method reusable?

Ans;A reusable method should perform one task and have a clear purpose.

Question 4

How does good naming improve software quality?

Ans :Good names make the code easier to read and maintain.

Question 5

If tomorrow the Placement Office changes its eligibility policy,

How should your design minimize the required code changes?

Keep the eligibility logic separate so changes are needed in only one place.

Sprint Retrospective

Every successful Agile sprint ends with reflection.

Spend fifteen minutes discussing the following questions with your pod.

What went well?

Identify one design decision your team is proud of.

Ans ;We completed the application workflow and validations successfully.

What was difficult?

Which concept required the greatest discussion?

Why?

Ans :The validation logic, because multiple conditions had to be handled correctly.

What would you improve?

If you had another two hours,

What changes would you make before releasing this software?

Ans ;We would improve validations, error handling, and testing.

One lesson to remember

Each team member should write one engineering lesson learned during today's sprint.

Do not write a programming concept.

Write an engineering principle.

Examples include

- Understand before implementing.
 - Keep responsibilities separate.
 - Good names improve readability.
 - Build incrementally.
 - Every method should solve one problem.
-

Tech Lead's Closing Note

Today you wrote your first business service in Apex.

Months from now, you may not remember every keyword or every syntax rule.

That is perfectly normal.

What matters is something much more valuable.

Today you learned that software is not created by typing quickly.

It is created by thinking carefully.

Every class exists because the business has a responsibility.

Every method exists because the business performs an activity.

Every variable represents information that the business cares about.

Every line of code should make the software more useful to its users.

Never measure yourself by the number of classes you write.

Measure yourself by the number of business problems you solve.

That is what professional software engineers do.

That is what great Salesforce developers do.

Chapter 4 Summary

Congratulations on completing Sprint 4.

In this chapter you have learned to:

- ✓ Understand the role of business logic in enterprise applications.
- ✓ Identify business rules before writing code.
- ✓ Organise software around business responsibilities.
- ✓ Design service classes using sound engineering principles.
- ✓ Create meaningful Apex classes and methods.
- ✓ Build your first business service using an incremental engineering approach.
- ✓ Appreciate that architecture and design precede implementation.

In the next sprint, you will learn how Salesforce retrieves and manipulates data efficiently using **SOQL and DML**, enabling your business services to interact intelligently with the Salesforce database.

Remember: *Code is not the goal. Well-designed software that solves real business problems is the goal.*